Product Requirements Document (PRD)

Lightweight Predictive Congestion Estimator for TCP

Project type	College networking project (simulation + machine learning)
Owner	Antra (24BDS0066)
Version	1.0 (draft)
Date	12 October 2026
Deadline	To be decided
Related document	`PROJECT_PLAN.md` (implementation plan and task order)

1. Summary

TCP finds out about congestion **late**: classic variants such as Tahoe and Reno, and even modern CUBIC, slow down only **after** packets have been lost. By then the router queue is full, delay is high, and lost data must be retransmitted.

This project builds a **lightweight machine-learning estimator** that predicts congestion about **0.5 seconds before it happens**, using only measurements the sender already has (RTT, throughput, congestion window and recent losses). The prediction is then used inside a new TCP variant, **Predictive TCP**, which eases off early. The project measures whether this early reaction reduces losses, retransmissions and delay compared with TCP Tahoe, Reno and CUBIC, while keeping throughput close to theirs.

2. Problem Statement

1. **Loss-based TCP reacts too late.** Tahoe, Reno and CUBIC treat packet loss as the main congestion signal, so the bottleneck queue must overflow before they react.
2. **Late reaction is costly.** It causes queueing delay (bufferbloat), bursts of drops, retransmissions, and sudden drops in the window (for example, Tahoe resets to 1 segment).
3. **Early signals exist but go unused.** RTT starts rising and the sender's own queued data grows *before* the queue overflows, but classic TCP does not use these signals to predict congestion.
4. **Heavy ML is unsuitable inside TCP.** Deep learning is too slow and complex to run per ACK. A predictor must be **small, fast and explainable**.

3. Goals and Non-Goals

Goals

- **G1.** Simulate and compare **TCP Tahoe, Reno and CUBIC** under low, medium and high congestion in NS-3.
- **G2.** Collect a clean dataset of **RTT, packet loss, throughput, congestion window, queue length and retransmissions**.
- **G3.** Train a **lightweight model** (Decision Tree or Logistic Regression) that classifies congestion as **Low / Medium / High**, and as a binary **congested / not congested**, **ahead of time**.
- **G4.** Build **Predictive TCP**, which uses the model's prediction to adjust its sending rate early.
- **G5.** Show a **measurable improvement** (fewer losses and retransmissions, lower delay) or a clearly useful prediction compared with the existing variants.
- **G6.** Produce graphs, a README, a report and a presentation suitable for academic evaluation.

Non-Goals

- **No deep learning** (no neural networks or reinforcement learning).
- **No real-network or hardware deployment.** Everything runs in simulation on a laptop.
- **No graphical app or web dashboard.** Results are shown with graphs and tables (a small results report is optional).
- **No changes to routers** (no AQM/ECN design). The mechanism is purely **sender-side**.
- **No support for other variants** (BBR, Vegas and so on), except as a mention in related work.

---

4. Stakeholders and Users

Stakeholder	Need
Student (project owner)	A working, explainable project that can be run, defended in a viva and extended
Faculty evaluator	Clear problem, correct methodology, reproducible results, honest analysis, and a measurable contribution
Future reader / student	A README and scripts to rerun every experiment with one command

---

5. Scope

In Scope

- An NS-3 dumbbell network with one shared bottleneck link and a FIFO queue.
- TCP Tahoe and Reno (implemented as custom NS-3 classes), TCP CUBIC (built into NS-3), and Predictive TCP (new).
- Three congestion levels, five random seeds each.
- A Python pipeline: dataset building, model training and evaluation, exporting the model to C++, and graphs.

- Documentation: README, report and presentation.

Out of Scope

- Wireless or mobile channels, multiple bottlenecks, mixed variants competing in the same run (possible future work).
- Real-time dashboards or online deployment.

---

6. Functional Requirements

6.1 Simulation (NS-3, C++)

ID	Requirement	Priority
FR-1	Simulate a dumbbell topology: N senders → R1 → **10 Mbps, 20 ms bottleneck** → R2 → receiver, with 100 Mbps access links.	Must
FR-2	Bottleneck uses a **FIFO queue (50 packets)** whose occupancy and drops can be traced.	Must
FR-3	Support TCP **Tahoe** (cwnd → 1 segment on loss, then slow start), **Reno** (fast retransmit + fast recovery, halving the window), and **CUBIC**.	Must
FR-4	Support **three congestion levels**: Low (~30% load), Medium (~90% bursty load), High (saturated + UDP background).	Must
FR-5	Be configurable from the command line: `--variant`, `--level`, `--seed`, `--simTime` (and bottleneck rate, delay, queue size).	Must
FR-6	Record **per flow, every 100 ms**: smoothed RTT, minimum RTT, cwnd, throughput, retransmissions and loss events; and at the bottleneck, queue length and drops. Write it to CSV.	Must
FR-7	Record an end-of-run **FlowMonitor** summary per flow (lost packets, delay, throughput).	Should
FR-8	Runs must be **reproducible**: the same seed gives the same results.	Must

6.2 Dataset and Machine Learning (Python)

ID	Requirement	Priority
FR-9	Build a dataset from the baseline runs with **sender-measurable features**: RTT ratio, RTT trend, estimated queued packets, cwnd, cwnd relative to its recent maximum, throughput, and loss events in the last 0.5 s and 2 s.	Must
FR-10	Label each sample from the **real bottleneck queue 0.5 s ahead**: High (drop or ≥ 80% full), Medium (≥ 40%), Low (otherwise), plus a binary label.	Must
FR-11	Train and compare **Decision Tree** and **Logistic Regression**, with training seeds kept separate from test seeds.	Must
FR-12	Report accuracy, per-class precision/recall/F1, a confusion matrix, feature importance and **early-warning time**.	Must
FR-13	**Export** the chosen decision tree automatically as a C++ header (`if/else` rules) for use inside NS-3.	Must

6.3 Predictive TCP (Proposed Mechanism)

ID	Requirement	Priority
FR-14	`TcpPredictive` computes the same features **inside the sender** every 100 ms and runs the exported model.	Must
FR-15	Act on the prediction: **Low** → normal growth; **Medium** → slow growth (25% of Reno's rate); **High** → reduce cwnd by **β (default 0.85)**, at most once per RTT. The predictor never acts below 4 segments.	Must
FR-16	Keep the normal TCP loss response as a fallback if loss still happens.	Must
FR-17	β and the prediction interval must be configurable (NS-3 attributes).	Should
FR-18	Log how many early (proactive) reductions were made per run.	Should

6.4 Analysis, Visualisation and Reporting

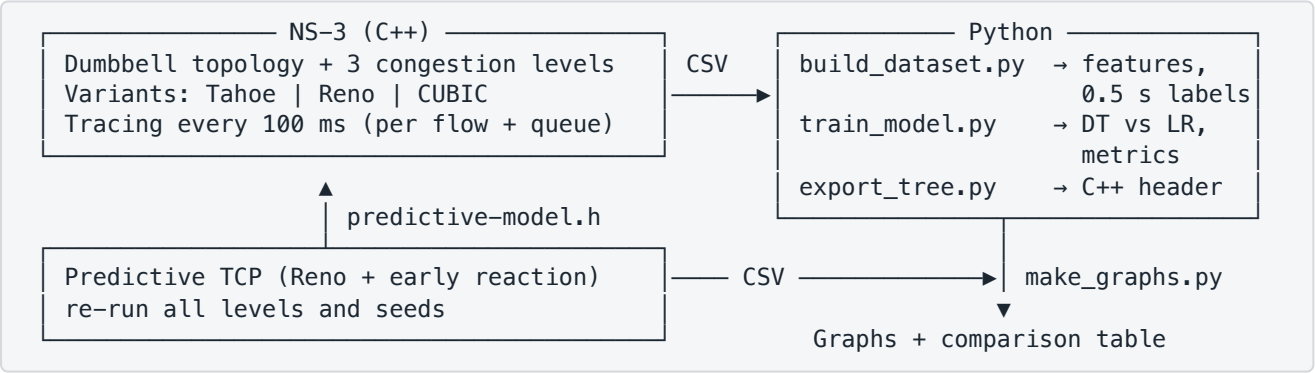
ID	Requirement	Priority
FR-19	Time-series graphs per level: **throughput, RTT, cwnd, queue length** for each variant.	Must
FR-20	Comparison bar charts (mean ± standard deviation over seeds): **throughput, RTT/delay, packet loss, retransmissions**, and the Jain fairness index.	Must
FR-21	ML graphs: confusion matrix, feature importance, tree diagram, early-warning histogram, Decision Tree vs Logistic Regression.	Must
FR-22	A final comparison table: **Tahoe vs Reno vs CUBIC vs Predictive** for each congestion level.	Must
FR-23	**One-command reproduction** (`run_all.sh`): build → simulate → train → export → re-simulate → graphs.	Should
FR-24	Optional small HTML/Markdown results summary ("dashboard").	Could

---

7. Non-Functional Requirements

ID	Requirement
NFR-1 **Lightweight**	Model depth ≤ 5 (≤ 5 comparisons per prediction); no ML library needed at run time; prediction runs at most once per 100 ms per flow.
NFR-2 **Deployability**	Uses **only sender-observable inputs**. The router's queue length is used only for training labels.
NFR-3 **Explainability**	The decision rules must be readable and explainable in a viva (tree diagram plus exported `if/else` code).
NFR-4 **Performance**	Each 60 s simulation completes in under ~1 minute on the laptop; the full experiment set completes in under ~1 hour.
NFR-5 **Footprint**	NS-3 ≈ 200 MB; all results and CSVs ≤ 1 GB (the laptop has limited free space).
NFR-6 **Reproducibility**	Fixed seeds, versions recorded (NS-3.48, Python 3.14, scikit-learn 1.9), and every number in the report traceable to a CSV.
NFR-7 **Code quality**	Medium difficulty, readable, and organised as described in the plan's folder structure.
NFR-8 **Honesty**	Limitations, thresholds and any trade-offs (such as lower throughput) are reported, not hidden.

8. System Overview



9. Data Requirements

Data	Granularity	Source	Used for
Smoothed RTT, minimum RTT	per flow, 100 ms	TCP socket trace	Features, delay graphs
Congestion window	per flow, 100 ms	TCP socket trace	Features, cwnd graphs
Throughput	per flow, 100 ms	Packet sink	Features, throughput graphs
Retransmissions, loss events	per flow, 100 ms	TCP socket traces	Features, loss graphs
Queue length, drops	bottleneck, 100 ms	Queue disc traces	**Labels**, queue graphs
Lost packets, delay	per flow, end of run	FlowMonitor	Summary table

Volume: about 45 baseline runs + 15 Predictive runs, which is about 10^5 samples in total. All data is stored as CSV.

---

10. Success Metrics and Acceptance Criteria

The numbers below are **targets**. The final report will state the values actually achieved.

10.1 Baseline Correctness

- □ **Tahoe:** cwnd traces show a reset to **1 segment** after a loss, followed by slow start.
- □ **Reno:** the window halves after 3 duplicate ACKs (fast recovery, no reset to 1).
- □ **CUBIC:** the characteristic cubic (concave/convex) window growth is visible.
- □ **Congestion levels:** the queue rarely fills at Low, is bursty at Medium, and stays near full with drops at High.

10.2 Prediction Quality (held-out seeds 4–5)

- □ 3-class **macro-F1 ≥ 0.70** — ❌ not met: 0.605 (Medium class F1 only 0.20; see README limitations)
- ☑ Binary **recall for "congested" ≥ 0.80** (few missed congestion events) — ✅ 0.863 (Phase 6 model)
- ☑ **Median early warning ≥ 200 ms** before an actual drop — ✅ median ≥ 1 s; 95% of drop bursts warned ≥ 200 ms ahead
- ☑ Decision Tree is at least as accurate as Logistic Regression, or the trade-off is explained — ✅ macro-F1 0.605 vs 0.540

10.3 Predictive TCP vs Reno and CUBIC (Medium and High levels)

- □ **≥ 30% fewer** bottleneck drops / retransmissions than Reno
- □ **Lower average RTT** (queueing delay) than Reno and CUBIC
- □ **Throughput ≥ 90%** of Reno's
- □ **Fairness index ≥ 0.9** between flows

10.4 Deliverables Complete

- □ All items in Section 12 present, and `run_all.sh` reproduces the graphs from scratch.

---

11. Milestones

#	Milestone	Output	Exit criterion
M0	Environment setup	NS-3.48, CMake/Ninja, Python venv	✅ Done
M1	Baseline simulation	Tahoe/Reno classes, simulation program, 45 runs	✅ Done (10.1 criteria met)
M2	Dataset + model	Features, labels, DT vs LR, exported header	✅ Done (10.2 partly met, documented)
M3	Predictive TCP	`TcpPredictive`, 15 runs, tuning of β	✅ Done (all 10.3 criteria met)
M4	Results	All graphs, comparison table, README, `run_all.sh`	✅ Done
M5	Stronger baselines	Vegas + BBR, 30 runs, 6-variant comparison	✅ Done
M6	Stronger ML model	Delay-trend features, horizon comparison, re-exported tree	✅ Done (10.2: 3 of 4 met; 10.3: all met)
M7	Delay-aware Predictive TCP	Delay reaction, tuning on seeds 1–3, re-runs	✅ Done (high-load RTT –37%, 5/5 seeds)
M8	Robustness tests	Buffer sweep, mixed Predictive + CUBIC	✅ Done (buffers: holds; vs CUBIC: 23% share, documented)
M8b	Competition detector	Detector in `TcpPredictive`, tuning on seeds 1–3	✅ Done (vs CUBIC 23% → 36%; 100-packet false alarms documented)
M9	Report + presentation	Final documents (format to be decided)	Ready to submit

12. Deliverables

1. NS-3 source: `tcp-predictor.cc`, `tcp-variants.h/.cc`, `predictive-model.h`
2. Python source: `build_dataset.py`, `train_model.py`, `export_tree.py`, `make_graphs.py`
3. Simulation results (CSV) and model outputs
4. Graphs and screenshots
5. `README.md` and `run_all.sh`
6. Project report
7. Presentation slides

13. Assumptions and Dependencies

- **Software:** NS-3.48 builds and runs on macOS with Apple clang 17 (already verified), and Python 3.14 with scikit-learn 1.9, pandas, NumPy and Matplotlib (installed).
- **Network model:** a single bottleneck with a FIFO queue is a fair and common model for comparing congestion control.

- **TCP options:** SACK is enabled and the minimum RTO is 1 s for all variants, so they are treated equally. (NS-3's CUBIC misbehaves without SACK; Tahoe's and Reno's window behaviour is set by the custom recovery classes and is unaffected.)
- **Labels:** the 40% and 80% queue thresholds and the 0.5 s horizon are reasonable starting values and may be tuned.

---

14. Risks and Mitigations

Risk	Impact	Mitigation
NS-3 has no Tahoe or classic Reno	Comparison is incomplete	Implement them as small custom recovery classes; validate with cwnd traces (10.1)
Class imbalance (too many "High" samples at saturation)	Biased model	Balanced class weights, macro-F1, and check the class distribution per level
Predictive TCP cuts throughput too much	Weak "improvement" claim	Tune β and the thresholds; report the throughput–delay trade-off honestly
Model overfits one scenario	Poor generalisation	Train and test on different seeds; ratio-based features; shallow tree
Limited disk space	Build or runs fail	Minimal NS-3 module build (done, 192 MB); compact CSVs
Deadline not fixed	Rushed results	Milestones M1–M3 are the priority; the optional dashboard (FR-24) is dropped first

---

15. Open Questions

1. **Deadline:** needed to schedule milestones M1–M5.
2. **Report and presentation format:** Word + PowerPoint, PDF, or another required template?
3. **Faculty requirements:** does the evaluator require a specific report structure (e.g. Abstract, Literature Review, Methodology)?
4. **Variants:** should NewReno or BBR be added as extra baselines? (Not planned; possible future work.)